USBD37: Register a new wagon in the database - Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD37**.

1. Descrição da User Story

- As a Planner, I want to register a new wagon in the database. All supported gauges must also be registered.

Critérios de Aceitação e Regras de Negócio:

- O modelo de vagão é identificado por um ID único.
- O sistema deve validar se o ID já existe na base de dados.
- O sistema deve impedir a duplicação de nomes de modelos, ignorando diferenças de maiúsculas/minúsculas e espaços extra (normalização).
- Todas as bitolas suportadas pelo vagão devem ser registadas na tabela de associação `Wagon_Model_Gauge`.
- A implementação deve ser feita através de uma procedure PL/SQL.
- **Conforme clarificação do professor**: Regista-se **apenas UM gauge** por vagão.

---

2. Explicação do Script (Procedure PL/SQL)

A lógica central da **USBD37** foi encapsulada na procedure `pr_register_wagon_model`. Esta abordagem permite centralizar as regras de negócio e garantir a integridade dos dados ao registar um novo modelo de vagão.

Detalhes da Implementação:

1. **Assinatura da Procedure:** A procedure aceita 14 parâmetros que descrevem as características técnicas do vagão (ID, nome, fabricante, dimensões, capacidades, etc.) e a bitola suportada.
2. **Validação de Dados Obrigatórios:** Verifica se os campos essenciais (`p_model_id`, `p_model_name`, `p_gauge_id`) são `NULL`. Lança exceção `-20030` em caso afirmativo.
3. **Verificação de ID Duplicado:** Consulta a tabela `Wagon_Model` para verificar se o ID já existe. Lança exceção `-20031` se duplicado.
4. **Normalização de Texto e Verificação de Nome Duplicado:**
    - O nome de entrada é normalizado usando `UPPER(TRIM())` para garantir que a verificação de duplicados é robusta.
    - Compara o nome normalizado com todos os registos existentes.
    - Lança exceção `-20033` se o nome já existir.
5. **Inserção na Tabela Principal:** Insere o novo modelo na tabela `Wagon_Model` com todos os parâmetros fornecidos, aplicando `TRIM` aos campos de texto.
6. **Registro da Bitola Suportada:** Insere um registo na tabela de associação `Wagon_Model_Gauge`, ligando o modelo à bitola especificada.
7. **Transação:** Realiza `COMMIT` em caso de sucesso ou `ROLLBACK` em caso de erro.

Nota Importante: Conforme clarificado pelo professor, esta user story regista **apenas UMA bitola** por vagão, apesar do enunciado original sugerir plural. A implementação está correta e alinhada com os requisitos do docente.

Código: `USBD37-script.sql`

```
CREATE OR REPLACE PROCEDURE pr_register_wagon_model(
    p_model_id      IN NUMBER,
    p_model_name    IN VARCHAR2,
    p_maker         IN VARCHAR2,
    p_num_bogies    IN NUMBER,
    p_bogies_type   IN VARCHAR2,
    p_length_mm     IN NUMBER,
    p_width_mm      IN NUMBER,
    p_height_mm     IN NUMBER,
    p_weight_tons   IN NUMBER,
    p_max_speed_kmh IN NUMBER,
    p_payload_tons  IN NUMBER,
    p_volume_m3     IN NUMBER,
    p_type_desc     IN VARCHAR2,
    p_gauge_id      IN NUMBER      -- CORREÇÃO: Apenas UM parâmetro para bitola
) IS
    v_count NUMBER;
BEGIN
    -- 1. Validação de parâmetros obrigatórios básicos
    IF p_model_id IS NULL OR p_model_name IS NULL OR p_gauge_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20030, 'USBD37: ID, Nome do Modelo e Bitola são obrigatórios.');
END IF;

    -- 2. Verificar se o ID do modelo já existe
SELECT COUNT(*) INTO v_count FROM Wagon_Model WHERE id = p_model_id;
IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20031, 'USBD37: O Modelo de Vagão ID ' || p_model_id || ' já existe.');
END IF;

    -- 3. Verificar se o Nome do modelo já existe (Case Insensitive)
    -- Feedback: "Comparar apenas os nomes diretamente não vale nada."
SELECT COUNT(*) INTO v_count
FROM Wagon_Model
WHERE UPPER(TRIM(model_name)) = UPPER(TRIM(p_model_name));

IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20033, 'USBD37: Já existe um modelo com o nome "' || p_model_name || '".');
END IF;

    -- 4. Inserir na tabela principal Wagon_Model
INSERT INTO Wagon_Model (
    id, model_name, maker, num_bogies, bogies_type, length_mm, width_mm,
    height_mm, weight_tons, max_speed_kmh, payload_tons, volume_m3, type_desc, gauge
) VALUES (
             p_model_id, TRIM(p_model_name), TRIM(p_maker), p_num_bogies, TRIM(p_bogies_type), p_length_mm, p_width_mm,
             p_height_mm, p_weight_tons, p_max_speed_kmh, p_payload_tons, p_volume_m3, TRIM(p_type_desc), p_gauge_id
         );

-- 5. Inserir na tabela de associação (Wagon_Model_Gauge)
-- Garante que a bitola é registada como suportada.
INSERT INTO Wagon_Model_Gauge (wagon_model_id, gauge_value)
VALUES (p_model_id, p_gauge_id);

DBMS_OUTPUT.PUT_LINE('Sucesso: Modelo ' || p_model_id || ' registado com a bitola ' || p_gauge_id);
COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

3. Explicação de Cada Teste

Para validar a User Story USBD37, foram desenvolvidos 3 cenários de teste que cobrem inserções válidas e violações de integridade.

Teste 1: Inserção Bem-Sucedida

Objetivo: Verificar se é possível registar um novo modelo de vagão válido que ainda não existe no sistema.

Descrição: O teste garante primeiro que não existem conflitos prévios e depois tenta inserir o modelo "Modelo Teste 8001" com todas as especificações técnicas e a bitola Ibérica (1668mm).

Resultado Esperado: A procedure deve executar sem erros, e ambos os registos (na tabela principal e na tabela de associação) devem ser criados.

```
 SET SERVEROUTPUT ON;
DECLARE
v_new_model_id NUMBER := 8001; -- ID Novo
    v_gauge_id     NUMBER := 1668; -- Bitola Ibérica (Existe no USBD32)
    v_count_m      NUMBER;
    v_count_g      NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD37: TESTE 1 (Sucesso) ---');

    -- Limpeza preventiva
DELETE FROM Wagon_Model_Gauge WHERE wagon_model_id = v_new_model_id;
DELETE FROM Wagon_Model WHERE id = v_new_model_id;

-- Executar Procedure
pr_register_wagon_model(
        p_model_id      => v_new_model_id,
        p_model_name    => 'Modelo Teste 8001',
        p_maker         => 'Fabrica Teste',
        p_num_bogies    => 4,
        p_bogies_type   => 'Y25',
        p_length_mm     => 14000,
        p_width_mm      => 3000,
        p_height_mm     => 4000,
        p_weight_tons   => 20,
        p_max_speed_kmh => 120,
        p_payload_tons  => 60,
        p_volume_m3     => 80,
        p_type_desc     => 'Contentor',
        p_gauge_id      => v_gauge_id
    );

    -- Validação
SELECT COUNT(*) INTO v_count_m FROM Wagon_Model WHERE id = v_new_model_id;
SELECT COUNT(*) INTO v_count_g FROM Wagon_Model_Gauge WHERE wagon_model_id = v_new_model_id AND gauge_value = v_gauge_id;

IF v_count_m = 1 AND v_count_g = 1 THEN
        DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Modelo criado e bitola associada.');
ELSE
        DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Dados não encontrados.');
END IF;

ROLLBACK; -- Deixa a BD limpa
END;
/
```

Teste 2: Falha - ID Duplicado

Objetivo: Verificar se o sistema impede a criação de um modelo com um ID que já existe na base de dados.

Descrição: O teste tenta registar um novo modelo usando o ID 1104, que já existe no sistema (inserido no USBD32).

Resultado Esperado: A procedure deve lançar a exceção `-20031` (ID Duplicado) e o teste deve capturá-la, confirmando que a validação funciona corretamente.

```
SET SERVEROUTPUT ON;
DECLARE
v_existing_id NUMBER := 1104; -- Já existe
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD37: TESTE 2 (Falha - ID Duplicado) ---');

    pr_register_wagon_model(
        p_model_id      => v_existing_id,
        p_model_name    => 'Nome Qualquer',
        p_maker         => 'Fabrica X',
        p_num_bogies    => 2, p_bogies_type => 'X', p_length_mm => 10, p_width_mm => 10, p_height_mm => 10, p_weight_tons => 10, p_m
        p_gauge_id      => 1668
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema permitiu duplicar ID.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20031 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de ID duplicado detetado corretamente.');
ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
END IF;
END;
/
```

Teste 3: Falha - Nome Duplicado (Case Insensitive)

Objetivo: Garantir que não são criados modelos com a mesma descrição, ignorando diferenças de maiúsculas/minúsculas e espaços extra.

Descrição: O teste tenta inserir um modelo com o nome " regmms 32 94 356 3 " (com espaços extra). Como o modelo com nome normalizado "REGMMS 32 94 356 3" já existe (ID 1104 do USBD32), o sistema deve detetar a semelhança semântica e bloquear a inserção.

Resultado Esperado: A procedure deve lançar a exceção `-20033` (Nome Duplicado), confirmando que a normalização e validação funcionam corretamente.

```
SET SERVEROUTPUT ON;
DECLARE
    -- Nome existente no USBD32 (ID 1104) com formatação diferente
v_duplicate_name VARCHAR2(100) := '  regmms 32 94 356 3  ';
    v_new_id         NUMBER := 8888;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD37: TESTE 3 (Falha - Nome Duplicado) ---');

    pr_register_wagon_model(
        p_model_id      => v_new_id,
        p_model_name    => v_duplicate_name,
        p_maker         => 'Fabrica X',
        p_num_bogies    => 2, p_bogies_type => 'X', p_length_mm => 10, p_width_mm => 10, p_height_mm => 10, p_weight_tons => 10, p_m
        p_gauge_id      => 1668
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema permitiu nome duplicado.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20033 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de Nome duplicado detetado corretamente.');
ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
END IF;
END;
/
```